Parallel Virtual Shared Memory, Large Data, and R

Bryan Lewis

Semi-professional R enthusiast
Cleveland, OH

Chicago RUG, August 2010

What is this?

Pvshm and PVFS2 let us memory map objects across file systems and/or RAM spanning Linux clusters.

And allow all the computers in the cluster fast parallel access to the objects.

And can be used from R with *indexing/mmap* and the *bigmemory* package families.

Some possibilities...

"Glue" the RAM across a cluster together into a huge shared parallel analog of `/dev/shm` available to each cluster node.

High-performance parallel file system access to cluster-wide storage accessible to non-HPC specialized applications that use mmap.

Parallel data analysis in R using `indexing` or `bigalgebra` when each process needs access to an entire large data set. Interesting possibilities include:

- The bootstrap
- Jacknife
- GCV
- Random forests

PVFS2

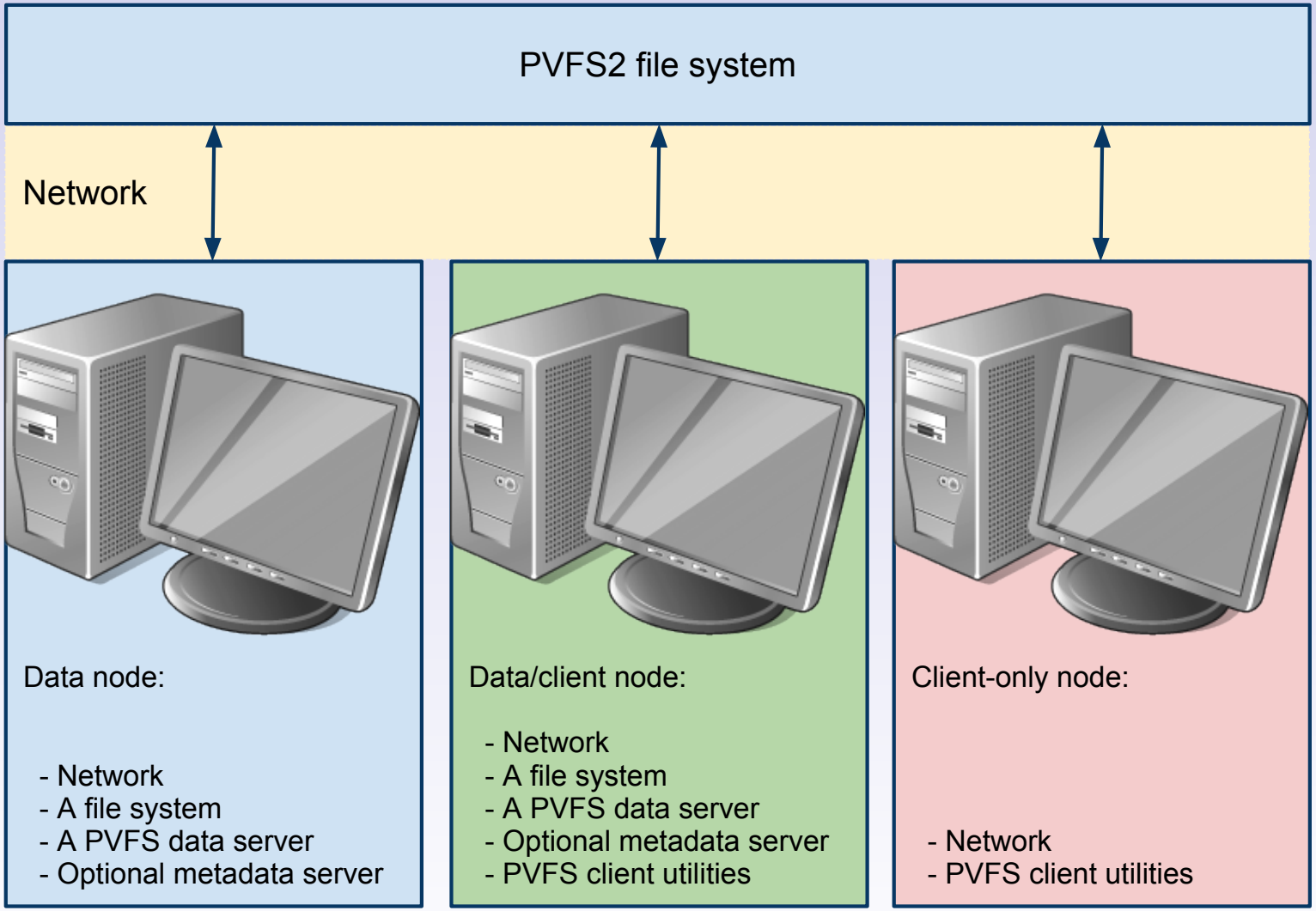
It's a scalable, high-performance parallel file system for Linux clusters developed at Clemson and ANL.

Effectively uses high-performance fabrics (IB, MX, 10GigE).

Mounted/accessed over the network or via MPI-IO.

PVFS2 implements a very basic file system, favoring speed and scalability over features: no locking, and no mmap.

PVFS2 Overview



Pvshm

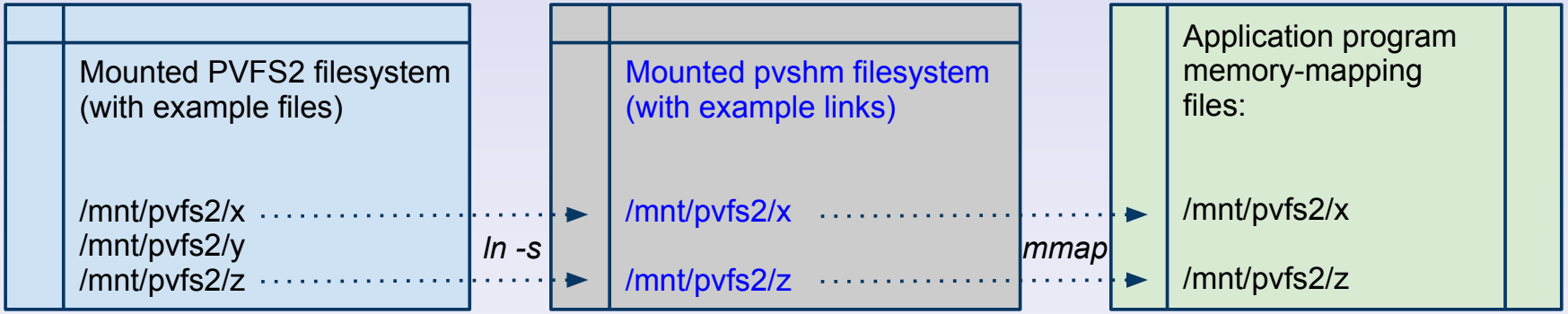
Pvshm is a strange Linux file system that converts address space operations to read and write operations on files living in other file systems.

Written in 2005 at Rocketcalc and recently updated for newer kernels. Quite similar to UML hostfs.

Does not enforce cache consistency, but does provide applications with tools to manage cache state themselves.

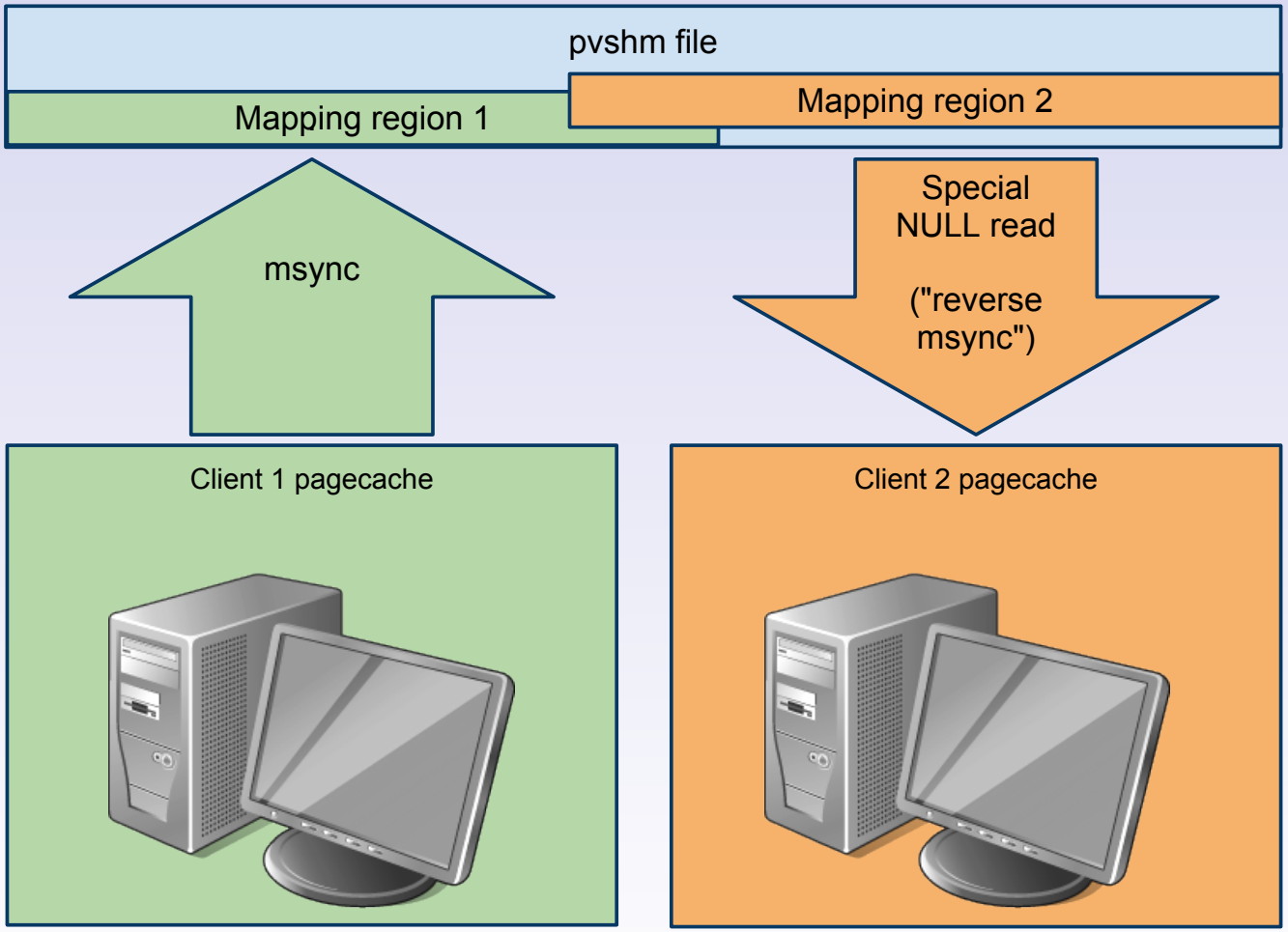
Overview

A special symbolic link operation registers files in pvshm and makes them available for memory mapping by applications.



Cache consistency tools

Not guaranteed atomic! So, clients need to coordinate among themselves.



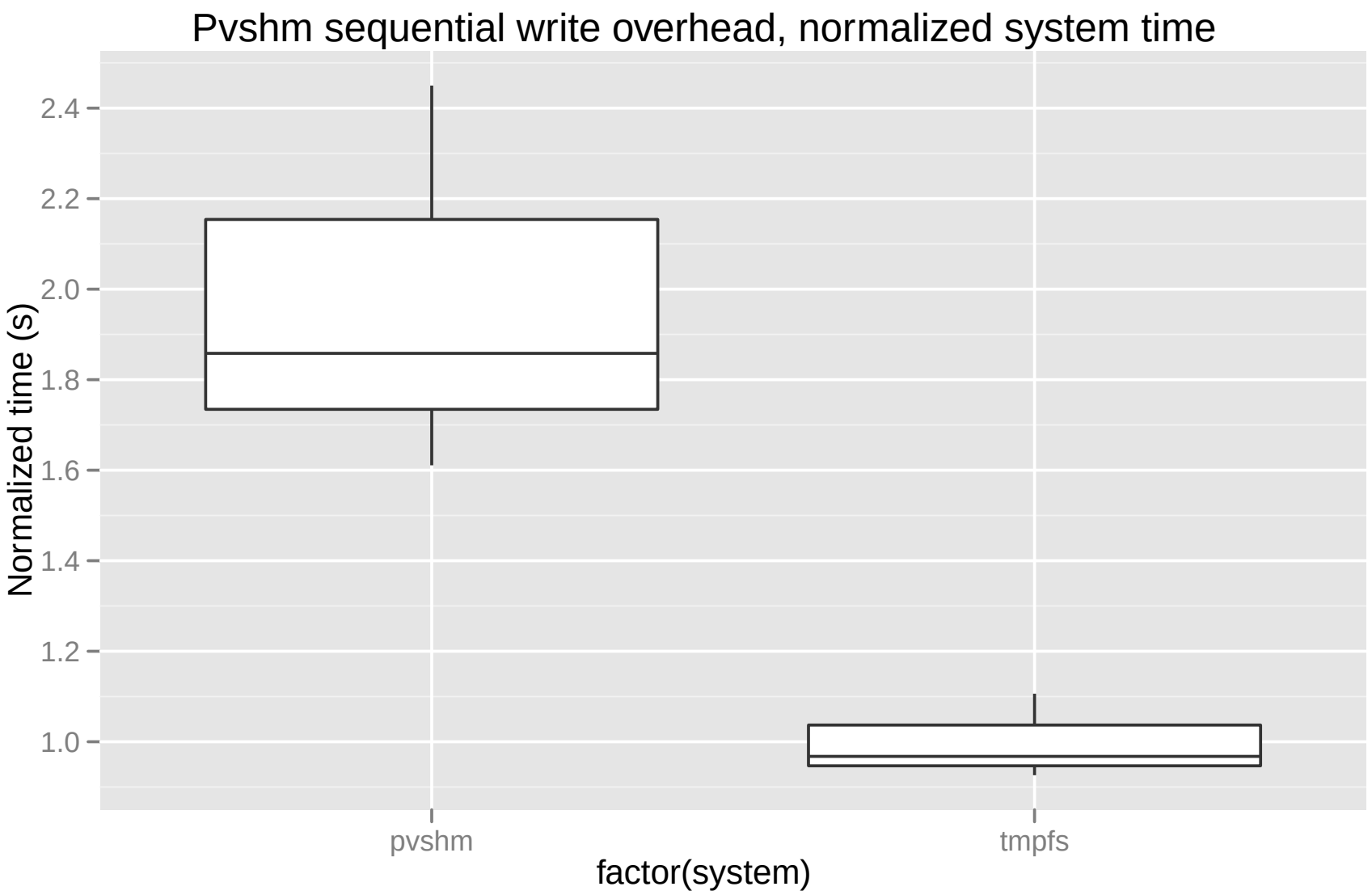
Amazon EC2 HPC Instances

$1.30 per hour buys you:

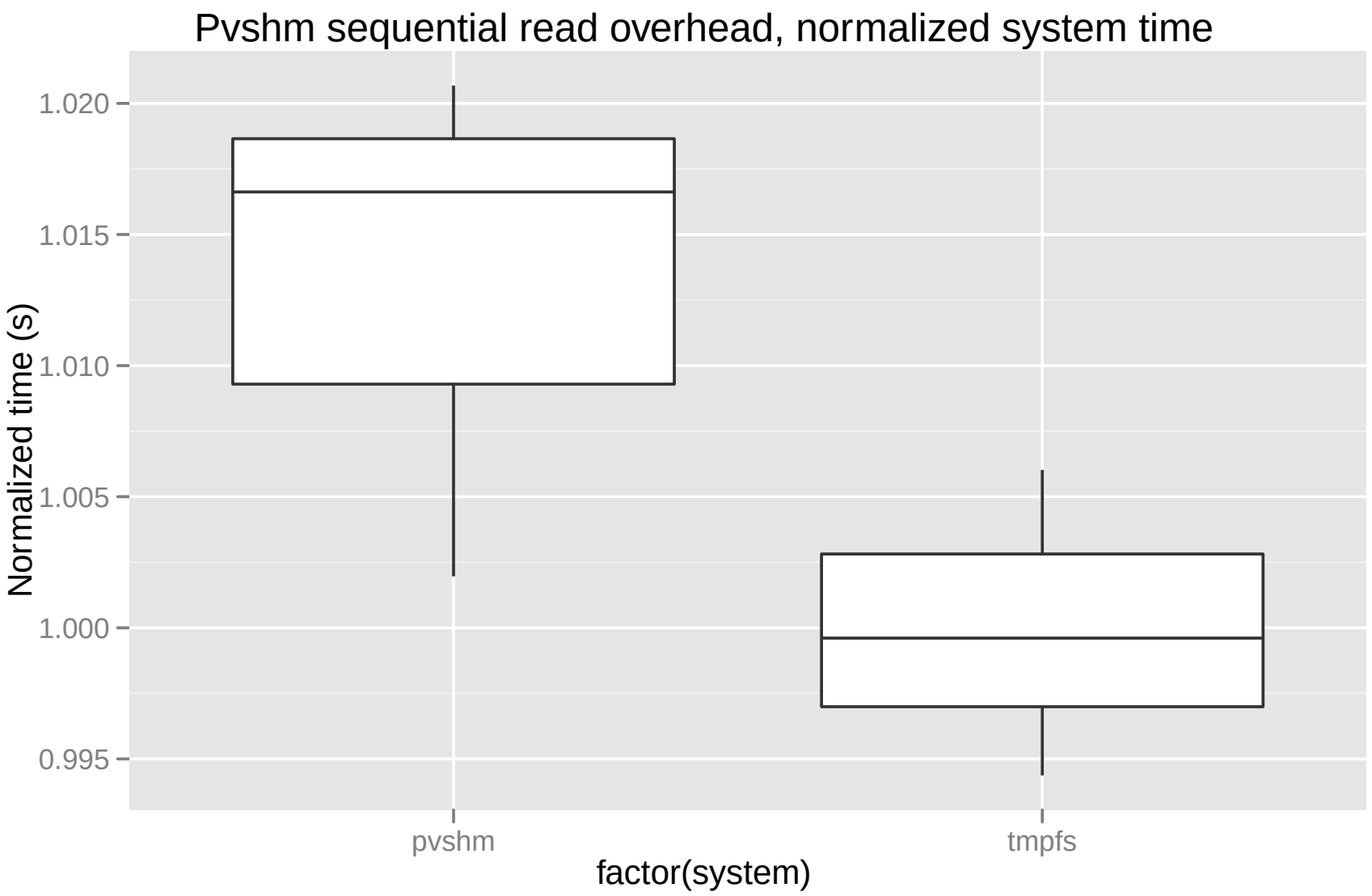
- Eight Intel Xeon X5570 cores at 2.93GHz
- 23GB RAM
- 10GB disk storage (cheap to add more)
- *Guaranteed* inter-node bandwidth of 10Gb/s*

* In theory, at least. I've had intermittent bandwidth issues.

Pvshm sequential write overhead versus RAM

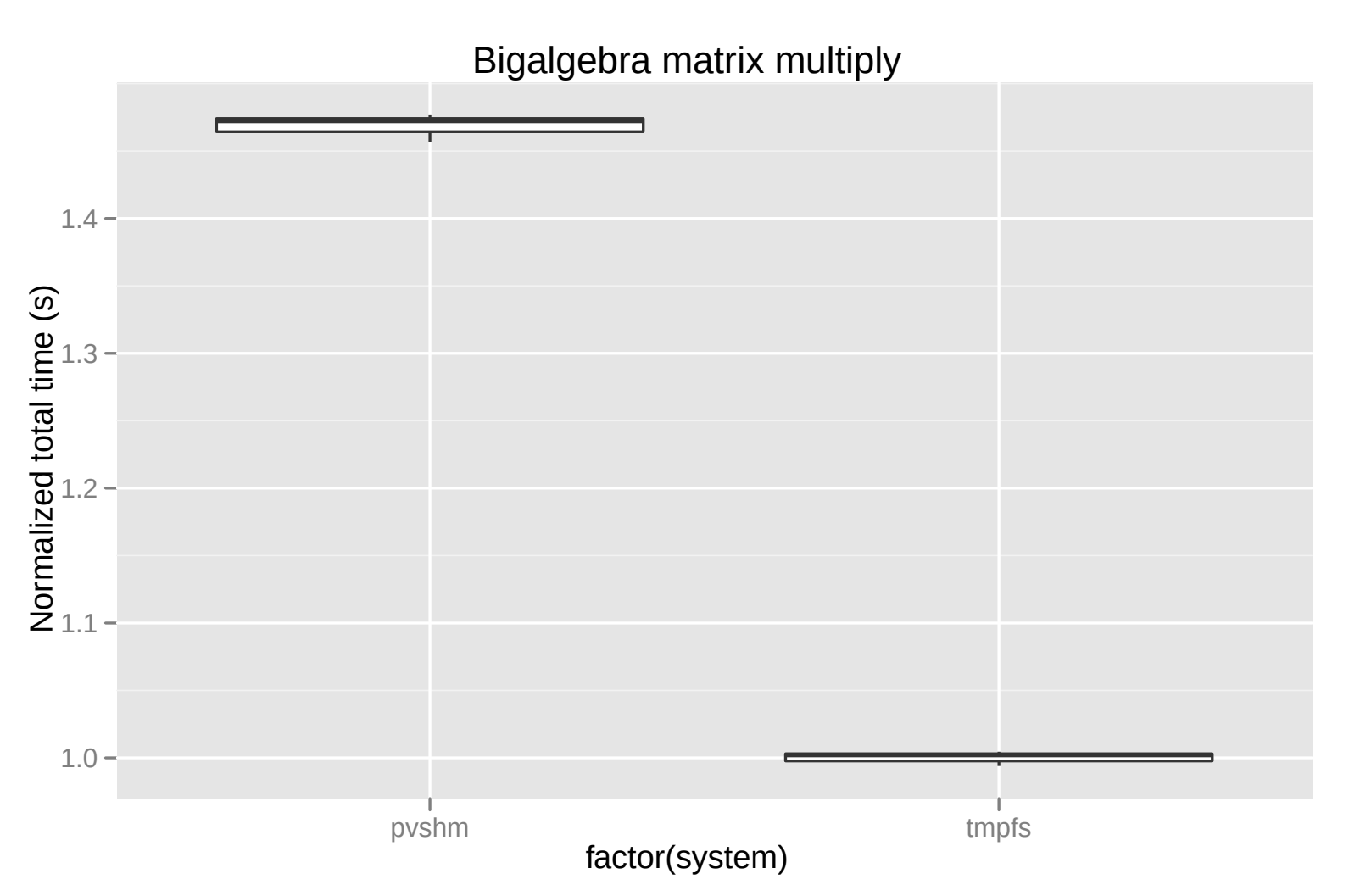


Pvshm sequential read overhead versus RAM



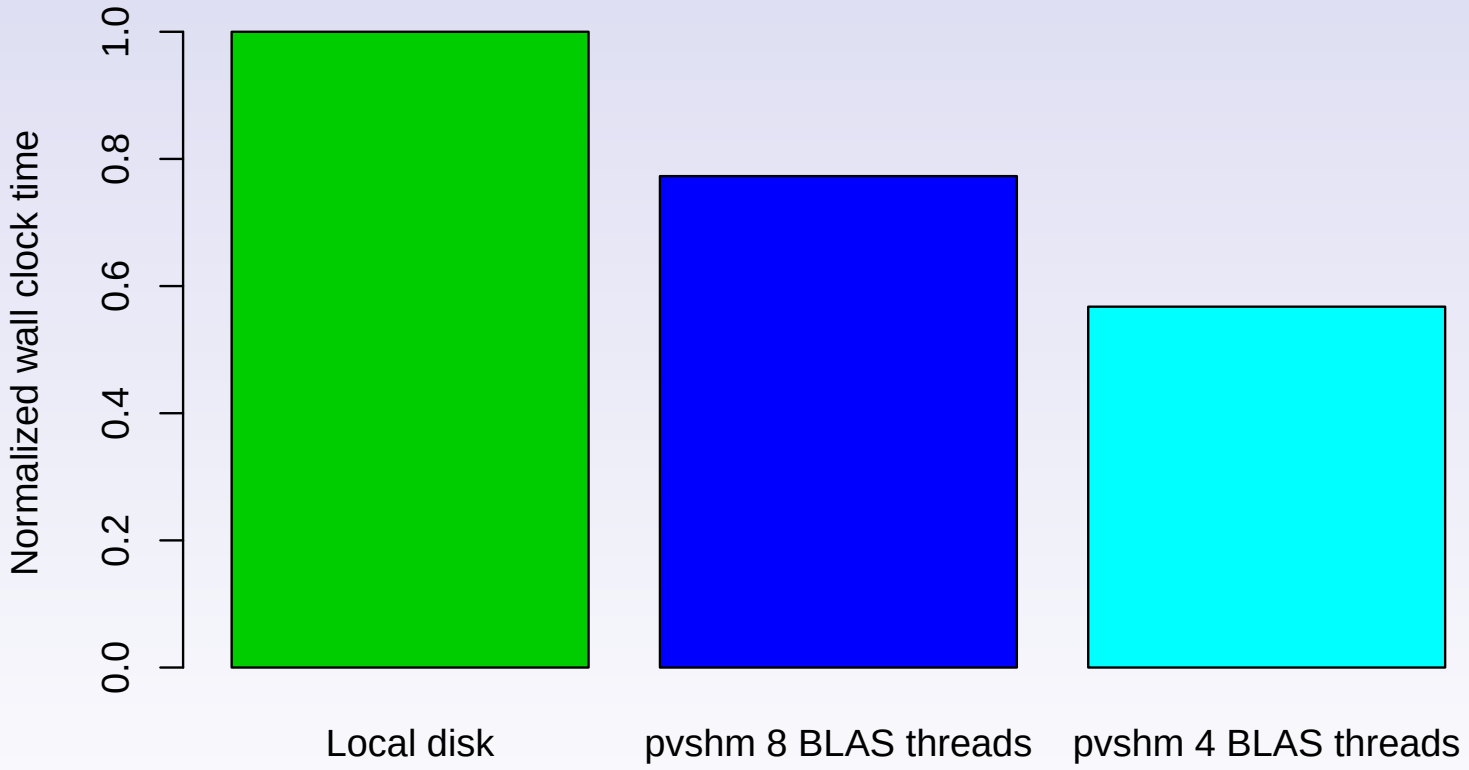
Pvshm bigalgebra overhead versus RAM

Compute product of $16,384 \times 16,384$ dense matrices.



Pvshm + PVFS2 (RAM) versus local disk

Compute product of $16,384 \times 16,384$ dense matrices, buffering prohibited (no free RAM), $2\times$ Amazon HPC EC2 nodes running PVFS2, bigmemory + bigalgebra, AMD ACML-4.4 BLAS.



Issues/Direction

- Move user space file access to user space daemon (cf FUSE). This should improve performance/reduce overhead.
- Standard file write operations are not yet supported. We plan to add pass-through standard write operations similar to the way read operations are presently implemented.
- Only compatible with very recent kernels. The Linux VFS API changes *very* frequently, making wide kernel compatibility hard.
- We will release an Amazon EC2 HPC AMI with PVFS2 and pvshm installed soon to make this easy to test and play with.